ARE REASONING MODEL BENCHMARK GAINS REAL? MEASURING THE INFLATIONARY EFFECT OF DECONTAMINATION FAILURES

Arno Demarteau **Rishbha Jain** **Victor Ngetich**

ABSTRACT

Open reasoning model post-training has produced striking gains on established mathematical benchmarks, yet the validity of these gains depends critically on the integrity of decontamination pipelines that are meant to prevent training-test overlap. We conduct a systematic audit of three prominent open post-training projects – s1 (Muennighoff et al., 2025), Tülu 3 (Lambert et al., 2025), and OpenThoughts (Guha et al., 2025) – each of which claimed decontamination against MATH-500 (Hendrycks et al., 2021) before release. Using a multi-stage detection pipeline combining n-gram replication, TF-IDF retrieval, dense semantic embeddings, and LLM-as-judge verification, we identify benchmark items that survived each project's decontamination despite meaningful overlap with training data. We then conduct behavioral analyses via perturbation testing and chain-of-thought structural comparison, finding that contamination manifests as solution-schema brittleness rather than verbatim recitation: contaminated problems produce a 37.5% answer-abandonment rate under numerical perturbation versus 20.8% for clean problems, alongside measurably fewer self-corrections and higher math token density in reasoning traces.

1 PROBLEM DEFINITION AND SCOPE

Post-training has become the primary mechanism by which open language models acquire reasoning capabilities. Projects such as s1 (Muennighoff et al., 2025), Tülu 3 (Lambert et al., 2025), and OpenThoughts (Guha et al., 2025) report substantial gains on MATH-500 (Hendrycks et al., 2021), GPQA Diamond (Rein et al., 2023), and AIME by fine-tuning on curated datasets of high-quality reasoning traces. These benchmark scores are widely cited as evidence that supervised post-training produces genuine capability improvements. The assumption underlying all such claims is that training data does not overlap with the test sets used for evaluation. This assumption is rarely verified.

Data contamination, the presence of evaluation benchmark items in training data, inflates reported performance and can invalidate scientific conclusions drawn from evaluation scores (Sainz et al., 2023). This threat is especially acute in the post-training setting: post-training corpora are orders of magnitude smaller than pretraining data, meaning a small number of contaminated items can have disproportionate influence. Critically, Kocyigit & Yildirim (2026) show in a controlled study that contamination injected during pretraining is not erased by continued training it is merely submerged, and is readily amplified by post-training, with SFT inflating scores specifically on contaminated benchmarks while GRPO produces smaller but broader gains. This establishes direct theoretical motivation for post-training-specific contamination audits.

All three projects we study applied decontamination before release. s1 and Tülu 3 used 8-gram token overlap matching; OpenThoughts combined fuzzy string matching with 13-gram overlap and reported a 99.6% true-negative rate on a manually constructed test set (Guha et al., 2025). Despite these efforts, our preliminary audit reveals benchmark items surviving in all three released datasets. This motivates our central question: do decontamination pipelines in open post-training fail in predictable, systematic ways, and does this failure measurably inflate reported benchmark gains?

2 DATASET / ENVIRONMENT ARTIFACT

2.1 AUDITED TRAINING DATASETS

We audit three released, post-decontamination training datasets against MATH-500 (Hendrycks et al., 2021): s1K (1,000 items), Tülu 3 SFT mixture math subset (84,312 items), and OpenThoughts-114K (113,957 items). All audited datasets are the publicly released, post-decontamination versions; any contamination we identify survived each project's own filter. The evaluation benchmark is MATH-500, which spans 7 subjects (Algebra, Counting & Probability, Geometry, Intermediate Algebra, Number Theory, Prealgebra, Precalculus) and difficulty levels 1–5.

2.2 CONTAMINATED AND CLEAN SPLITS

After running the multi-stage detection pipeline (described in Section 5), we curate a contaminated split and a matched clean split for each model. Clean problems were drawn from a verified uncontaminated pool. For each model, the subject-by-level distribution of clean problems was matched exactly to the contaminated split to avoid confounding by difficulty, verified via programmatic comparison. Subject-by-level matching was enforced algorithmically; formal balance tests were not computed due to the small per-cell sample sizes.

Table 1: Evaluation set composition after contamination detection and clean-split curation.

Model	**Contaminated**	**Clean**	**Total prompts**	
OpenThoughts	24	24	144	$(24 \times 3 \times 2)$
Tulu	32	32	192	$(32 \times 3 \times 2)$

Average difficulty level: OpenThoughts 3.50, Tulu 3.28.

2.3 PRELIMINARY CONTAMINATION COUNTS

Table 2 summarizes preliminary contamination findings across all three projects before behavioral analysis.

3 BASELINES

We evaluate two open-weight instruction-tuned models (Table 3) under a within-model counterfactual design: the clean split for each model serves as the control, so per-model capability differences do not confound the contamination estimate. Raw accuracy on unperturbed original problems (Table 4, Original column) provides the zero-shot reference: 75–79% for OpenThoughts, 25–31% for Tulu. The heuristic detection baseline is each project's own n-gram filter; Table 2 shows what those filters missed and what our multi-stage pipeline recovered.

Shared sampling hyperparameters: $T = 0.8$, $N_{\text{samples}} = 1$, max new tokens $= 4{,}096$.

4 LITERATURE REVIEW

4.1 BENCHMARK DATA CONTAMINATION

Contamination has been recognized as a systemic threat to LLM evaluation since GPT-3 shipped with a filtering bug that contaminated several benchmarks (Brown et al., 2020). Sainz et al. (2023) provide the foundational taxonomy of contamination types and argue that NLP evaluation is structurally compromised when test data enters training corpora. The problem extends beyond pretraining: Kocyigit & Yildirim (2026) show that contamination resurfaces during SFT and GRPO post-training, with SFT producing benchmark-specific inflation. LiveBench (White et al., 2024) represents the strongest benchmark-side defense, drawing questions from recent competitions and papers with monthly updates, but cannot retroactively validate gains already reported on static benchmarks. Sun et al. (2025) rigorously evaluate 20 mitigation strategies and find none simultaneously achieves

Table 2: Preliminary contamination audit results across three open post-training projects against MATH-500. All findings are on released, post-decontamination datasets.

Project	Filter Claimed	Items Audited	Leaks Found
s1	8-gram overlap	1,000	7 flagged
Tülu 3	8-gram, 50% token match	84,312	53 flagged
OpenThoughts	Semantic (τ=0.85)	113,957	19 flagged
OpenThoughts	Semantic (τ=0.70) + LLM judge	113,957	127 flagged

Table 3: Model configurations.

Model	Variant	Precision	Max tokens
OpenThoughts	OpenThinker-7B	bf16	4,096
Tulu	Tülu-3-8B-SFT	bf16	4,096

high fidelity and contamination resistance, underscoring that detection and mitigation remain open problems.

4.2 Detection Methods

With corpus access, n-gram overlap is the dominant method (Brown et al., 2020), but structurally cannot detect paraphrase- or template-level contamination – a limitation Tülu 3 itself acknowledges (Lambert et al., 2025). LLM-as-judge approaches (Yang et al., 2023; Deng et al., 2024) offer greater semantic sensitivity but require precision validation. Without corpus access, black-box methods have proliferated: Min-K% (Shi et al., 2023) flags anomalously high token probabilities, TS-Guessing (Deng et al., 2024) tests whether models reconstruct masked benchmark content, CoDeC (Zawalski et al., 2025) uses in-context learning, showing that contaminated datasets reduce model confidence when used as context, achieving near-perfect dataset-level AUC. Our approach differs from all of these: we have direct access to released training datasets on HuggingFace, enabling item-level audit rather than dataset-level statistical inference, more actionable and more informative for characterizing failure mechanisms.

4.3 Reasoning vs. Memorization

Carlini et al. (2021) establish that LLMs memorize training data at scale and that memorized content is extractable through targeted prompting. He et al. (2025) extend this to benchmark evaluation, finding that 94.9% of MMLU mathematics problems are classified as "seen-like" in recent models via retrieval-augmented detection, suggesting surface decontamination leaves a large class of memorized items undetected. Behavioral signatures of memorization in chain-of-thought models have received less direct study. Our analyses test whether contaminated items elicit shorter, lower-variance reasoning traces and higher perturbation sensitivity—the behavioral fingerprint of recitation rather than computation.

5 Methodology

5.1 Multi-Stage Contamination Detection Pipeline

Our three-stage detection pipeline identifies contaminated items in each released training dataset:

1. **N-gram replication.** We replicate each project's exact filter using the Qwen2-7B tokenizer: 8-gram with $>$0 shared n-grams for s1, 8-gram with $>$50% token match for Tülu 3, 13-gram for OpenThoughts. Items that should have been removed by each project's own criteria but remain in the released dataset constitute our Type 1 (implementation failure) candidates.

2. **Semantic retrieval.** For each MATH-500 problem, we retrieve the top-k most similar training items using TF-IDF cosine similarity (Tülu 3) or dense sentence embeddings with FAISS (`all-mpnet-base-v2`, threshold 0.75–0.85; OpenThoughts). Candidates with zero n-gram overlap constitute our Type 2 (design failure) candidates.
3. **LLM-as-judge verification.** Candidate pairs are submitted to Gemini 2.5 Flash (Tülu 3) or GPT-4o-mini (s1) and classified as CONTAMINATED, RELATED, or CLEAN based on whether the same mathematical insight is required and whether training on the item confers meaningful advantage. Outputs are validated on a stratified sample to estimate precision.

5.2 Primary Metrics

Our primary evaluation metric is judge-confirmed contamination count per project. Secondary metrics are judge precision (proportion of TF-IDF candidates confirmed by the LLM judge) and false positive rate (proportion rejected), estimated via manual validation on a stratified sample per project.

For causal attribution of benchmark score inflation, the primary metric is the difference-in-differences (DiD) estimate (Section 5, DiD Framework). Secondary behavioral metrics include null rate (fraction of prompts with no extractable answer), chain-of-thought feature differences (word count, self-correction frequency, uncertainty marker frequency), and recitation rate (verbatim reproduction of pre-swap answers).

5.3 Behavioral Analysis

Confirming decontamination failure is necessary but not sufficient to show benchmark gains are inflated. We require behavioral evidence that models recite rather than reason on contaminated items. We formally define recitation as follows: a model M recites item x if and only if its accuracy degrades beyond threshold θ under reasoning-invariant perturbation $\rho(x)$, where ρ preserves logical structure but alters surface features:

$$\textit{Recitation}(M, x) = \mathbf{1}\left[\,\text{Acc}(M, x) - \text{Acc}(M, \rho(x)) > \theta\,\right] \tag{1}$$

We conducted three inference-only analyses on publicly available 7B–8B post-trained models: (1) **Perturbation testing**: number swap (changing numerical values while preserving mathematical structure) and surface noise (appending an irrelevant sentence) applied to both contaminated and clean splits, with accuracy degradation measured via the DiD estimator; (2) **CoT structural analysis**: comparing trace length, self-correction frequency, uncertainty markers, hedging, and math token density on contaminated vs. matched clean items on original prompts; (3) **Mechanistic probing**: layer-wise attention entropy and logit lens rank measured on a subset of OpenThoughts traces to test for earlier answer prediction on contaminated problems. Temperature-sampling variance (N=10 at T=1.0) was not completed at the current sample size and remains future work.

5.4 DiD Framework

We propose a **difference-in-differences (DiD)** framework to causally attribute benchmark score inflation to training data contamination. The key challenge in contamination detection is separating a model's genuine reasoning ability from its ability to recall memorized answers. A model that has seen a benchmark problem during training may answer it correctly not because it understands the underlying mathematics, but because it has learned to pattern-match the problem and retrieve its answer from weights.

Our design exploits two sources of variation simultaneously. First, we partition benchmark problems into a *contaminated* split (problems we have evidence appear in each model's training data) and a *clean* split (problems we have verified are absent from training). Second, for each problem in both splits, we generate two *perturbations*: a **number swap** (changing numerical values while preserving mathematical structure) and a **surface noise** addition (appending an irrelevant real-world sentence that does not alter the mathematical content or answer). A model that has memorized contaminated problems should be disproportionately sensitive to number swaps—it will fail when numbers change because its shortcut is disrupted—whereas a model that reasons from first principles should be equally robust to this perturbation in both splits.

The DiD estimator isolates the contamination effect:

$$\widehat{\text{DiD}} = \underbrace{\left(\bar{a}_{C,\text{orig}} - \bar{a}_{C,\text{perturb}}\right)}_{\Delta_C} - \underbrace{\left(\bar{a}_{N,\text{orig}} - \bar{a}_{N,\text{perturb}}\right)}_{\Delta_N} \tag{2}$$

where $\bar{a}_{C,\cdot}$ and $\bar{a}_{N,\cdot}$ are mean per-problem accuracies in the contaminated and clean splits respectively, and "perturb" refers to either `number_swap` or `surface_noise`. If contamination causes memorization, $\Delta_C > \Delta_N$ and $\widehat{\text{DiD}} > 0$: contaminated problems are more sensitive to perturbation. If contamination has no effect, both splits degrade equally and $\widehat{\text{DiD}} \approx 0$.

5.5 Implementation Details

Contamination identification. Contaminated problems were identified via the multi-stage pipeline described above (n-gram replication, semantic retrieval, LLM-as-judge verification) and stored as a pre-compiled CSV (`contaminated_and_perturbations.csv`) prior to inference. The contaminated set differs per model: 24 problems for OpenThoughts and 32 for Tulu, reflecting the differing contamination rates found in each project's released training data. An optional annotation file (`Annotation_CS288_Final.csv`) provides per-problem similarity scores and Type A/B contamination labels used for subgroup analyses.

Perturbation generation. Both perturbation types were generated using GPT-4o with structured prompts (see Appendix A for full system prompts).

- *Number swap:* GPT-4o was instructed to change numerical values to different but similarly-scaled numbers, preserve the mathematical structure and operation type, ensure the problem remains well-defined with a clean answer, solve the modified problem, and rate answer confidence as `high`, `medium`, or `low`. Output was constrained to JSON with keys `perturbed_problem`, `perturbed_answer`, `confidence`.
- *Surface noise:* GPT-4o was instructed to add exactly one non-mathematical context sentence (a setting, character motivation, or real-world scenario) without altering any existing wording, numerical values, or mathematical structure. The answer is unchanged by construction. Output was constrained to JSON with key `perturbed_problem`.

Both perturbation types are applied to problems in both splits, yielding a 2 (split) $\times$ 3 (perturbation type: original, number_swap, surface_noise) design per model.

Inference and evaluation. Clean problems were drawn from a verified uncontaminated pool, matched to the contaminated split by subject $\times$ level. All models were run on a single NVIDIA RTX PRO 6000 Blackwell (102 GB VRAM) in bfloat16 at T=0.8, N=1, max 4,096 tokens. Answers were extracted via the final `\boxed{}` expression and normalized; a GPT-4o-mini judge then re-evaluated equivalent-form answers (e.g., $\frac{1}{2}$ vs. 0.5), changing 25 OpenThoughts and 5 Tulu labels. Judge reliability was validated against human annotations: $\kappa = 0.83$. Per-problem accuracy differences $\delta_i = \mathbf{1}[\text{correct}_{\text{orig}}] - \mathbf{1}[\text{correct}_{\text{perturb}}]$ were aggregated into the DiD estimate; uncertainty was quantified via 10,000-iteration cluster bootstrap (95% CIs) and Welch's t-test.

5.6 Theoretical Framing

The DiD design provides a valid causal estimate of the contamination effect under the **parallel trends assumption**: in the absence of contamination, contaminated and clean problems would show equal sensitivity to perturbation. This assumption is plausible given that the clean baseline was curated to match the contaminated split's subject-level distribution, and that both perturbation types are mathematically preserving (the correct answer is unchanged or independently recomputed).

The estimator is conservative in two senses. First, with $N = 1$ per prompt, per-problem accuracy is binary, so $\delta_i \in \{-1, 0, 1\}$; this inflates variance and reduces power relative to a continuous outcome. Second, surface noise — which does not change the answer — should produce $\Delta \approx 0$ in both splits under any model, making it a useful specificity check: a large positive surface-noise DiD would indicate a methodological artifact rather than contamination.

6 MAIN RESULTS

Table 4 reports LLM-as-judge accuracy across all conditions.

Table 4: Accuracy by model, split, and perturbation type (LLM-as-judge).

Model	Split	Original	Number swap	Surface noise
OpenThoughts	Contaminated	75.0% (18/24)	37.5% (9/24)	66.7% (16/24)
	Clean	79.2% (19/24)	33.3% (8/24)	79.2% (19/24)
Tulu	Contaminated	31.2% (10/32)	28.1% (9/32)	28.1% (9/32)
	Clean	25.0% (8/32)	25.0% (8/32)	18.8% (6/32)

Table 5 reports the DiD estimates, bootstrap confidence intervals, and Welch's t-test results.

Table 5: Difference-in-differences estimates. $\Delta = \bar{\delta}_{\text{orig}} - \bar{\delta}_{\text{perturb}}$ (mean per-problem accuracy drop). DiD $= \Delta_C - \Delta_N$. 95% CIs from 10,000-iteration cluster bootstrap. Welch's t-test.

Model	Perturbation	Δ_C	Δ_N	DiD	95% CI	t	p
OpenThoughts	Number swap	$+0.375$	$+0.458$	-0.083	$[-0.375, +0.208]$	-0.575	0.568
OpenThoughts	Surface noise	$+0.083$	$+0.000$	$+0.083$	$[-0.125, +0.333]$	$+0.700$	0.488
Tulu	Number swap	$+0.031$	$+0.000$	$+0.031$	$[-0.219, +0.281]$	$+0.239$	0.812
Tulu	Surface noise	$+0.031$	$+0.062$	-0.031	$[-0.281, +0.219]$	-0.255	0.799

No DiD estimate reaches statistical significance; all 95% confidence intervals span zero. The directional pattern for OpenThoughts number swap is *negative* (DiD $= -0.083$), suggesting contaminated problems are slightly *less* sensitive to number swap than clean ones—opposite to the memorization prediction—though this is statistically indistinguishable from zero. Projecting the DiD estimates to the full MATH-500 benchmark via Inflation $= (N_{\text{contaminated}}/500) \times$ DiD yields point estimates $\leq \pm 0.4$ pp with confidence intervals spanning ± 1.8 pp; the full inflation table and accuracy breakdown figure appear in Appendix E.

7 ABLATIONS

7.1 ABLATION 1: NULL RATE AS A COMPLEMENTARY METRIC

A model that cannot adapt when numbers change may fail to produce a boxed answer entirely, rather than producing a wrong one. We track this **null rate**—the fraction of prompts where no `\boxed{}` expression was found in the output (Table 6). This ablation tests what signal is lost when relying on accuracy alone versus accuracy combined with null rate as a contamination indicator.

Table 6: Null rates (no extractable answer) by model, split, and perturbation type.

Model	Split	Perturbation	Null rate
OpenThoughts	Contaminated	Number swap	**37.5%**
	Contaminated	Surface noise	20.8%
	Contaminated	Original	16.7%
	Clean	Number swap	20.8%
	Clean	Surface noise	20.8%
	Clean	Original	16.7%

The most striking signal is the **OpenThoughts contaminated number-swap null rate of 37.5%**—nearly double the clean counterpart (20.8%) and the highest null rate across all conditions. When numbers change on contaminated problems, OpenThoughts is substantially more likely to produce an incomplete or unfinished response than to reason from scratch. This contamination-induced

brittleness is invisible to standard accuracy metrics. Tulu shows a milder version of the same pattern (6.2% contaminated vs. 0.0% clean on number swap).

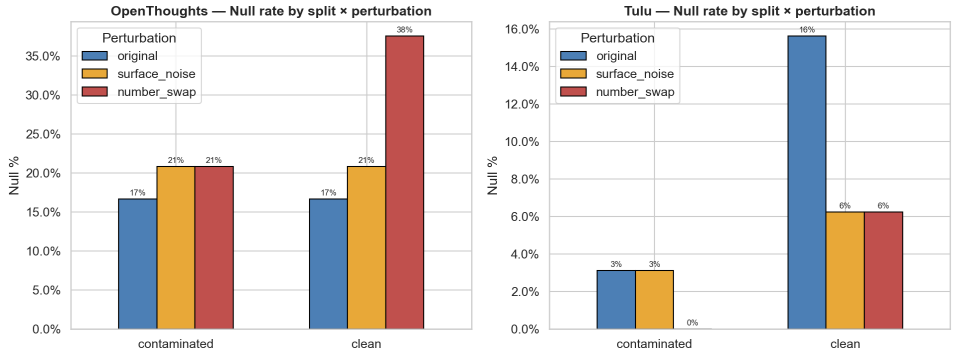

Figure 1: Null rates (no extractable \boxed{} answer) by model, split, and perturbation type. The contaminated number-swap bar for OpenThoughts (37.5%) is the clearest contamination signal in the study.

7.2 Ablation 2: Recitation vs. Shortcut Hypothesis

We tested whether models verbatim recited the original (pre-swap) answer on number-swapped problems. Neither model shows any recitation on contaminated problems: 0/14 evaluable responses for OpenThoughts (9 null, 1 ambiguous out of 24) and 0/24 for Tulu. The single recitation occurs in the clean split (1/18 for OpenThoughts), suggesting coincidence. Full breakdown in Appendix E. Contamination therefore operates via a *shortcut pattern*—a bias toward familiar solution trajectories—rather than rote key-value recall.

7.3 Ablation 3: Chain-of-Thought Behavioral Fingerprint (OpenThoughts)

OpenThoughts produces long chain-of-thought (CoT) traces ($\sim$1,200–1,500 words), enabling a quantitative analysis of reasoning style across splits. We measure six CoT features: word count, uncertainty marker frequency, hedging frequency, self-correction frequency, math token density, and type-token ratio (TTR). This ablation tests whether behavioral features within the reasoning trace provide a contamination signal that complements the DiD accuracy metric.

Table 7 reports features on *original*-perturbation prompts only, isolating the contamination signal from any difficulty introduced by the perturbations themselves.

Table 7: CoT features for OpenThoughts, original perturbation only. Δ = Contaminated − Clean.

Split	N	**Words**	**Uncertainty**	**Hedging**	**Self-corr.**	**Math%**
Clean	24	1,251	2.00	1.17	0.38	36.8%
Contaminated	24	1,160	1.33	0.67	0.17	45.2%
Δ (%)		-7%	-34%	-43%	$\mathbf{-55\%}$	$+23\%$

Contaminated traces are shorter and show substantially fewer markers of active reasoning: uncertainty expressions (-34%), hedging (-43%), and self-corrections (-55%). Math token density is higher ($+23\%$), consistent with more direct answer retrieval. Mann-Whitney U tests on per-problem feature values yield marginal significance for self-correction ($p = 0.098$) and math token density ($p = 0.093$); all other features are non-significant ($p > 0.15$). These effects do not survive multiple-comparison correction and should be treated as exploratory.

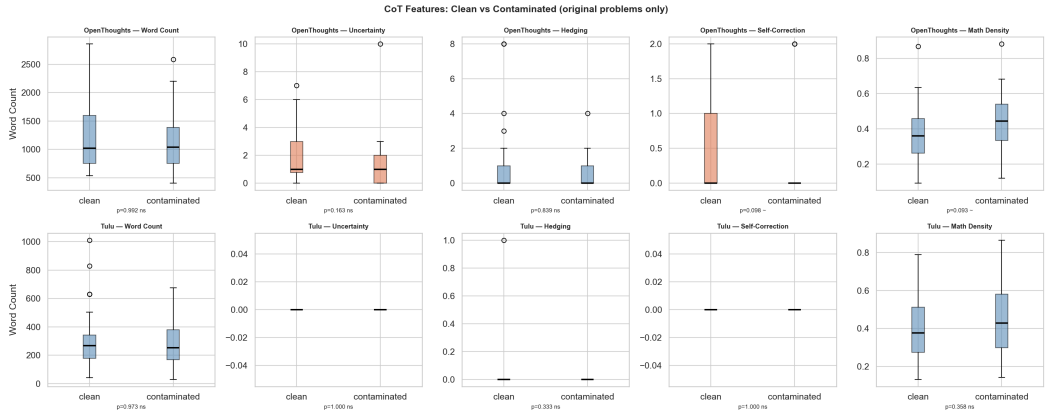

Figure 2: CoT feature distributions for OpenThoughts (original perturbation, contaminated vs. clean). Contaminated traces show reduced uncertainty, hedging, and self-correction, with higher math token density—consistent with more direct solution retrieval.

Tulu's traces are too terse ($\sim$280–315 words, with effectively zero uncertainty or self-correction markers) to yield a meaningful CoT signal; all Tulu CoT comparisons are non-significant.

7.4 Ablation 4: Mechanistic Probing and CoT Classifier (OpenThoughts)

A layer-wise mechanistic probe (attention entropy and logit lens rank of the correct answer token across 18 subject–level buckets) shows directionally consistent but highly variable signals: clean traces average $+0.007$ nats higher attention entropy and the correct answer token ranks $\sim$5,295 positions later, consistent with more generative processing on clean problems. Results are based on single forward passes ($N{=}1$) and are inconclusive at this sample size; full breakdown in Appendix C. A logistic classifier trained on CoT features (word count, math density, self-correction, hedging, uncertainty) achieves LOO-CV AUC ≤ 0.51—at or below chance—confirming that per-trace contamination classification is not feasible at the current sample size; ROC curves and feature coefficients in Appendix D.

8 Error Analysis

We identify four sources of error that limit the current study and point toward specific fixes.

Low statistical power. With $N{=}1$ sample per prompt and 24–32 contaminated problems per model, per-problem accuracy is binary and DiD variance is high. All confidence intervals span zero, but the secondary signals (37.5% null rate asymmetry, -55% self-correction) are directionally consistent with a real effect. A replication at $N{\geq}5$ with majority-vote scoring—already implemented in the pipeline but not yet run—would narrow the intervals enough to confirm or rule out inflation of practical magnitude (≥ 0.5 pp). This is the highest-priority extension.

Treatment assignment noise. Contamination labels come from the multi-stage pipeline (n-gram replication, embedding retrieval, LLM-as-judge), which is imperfect. Mislabelling a clean problem as contaminated attenuates the DiD toward zero (conservative bias); the reverse inflates it. We believe the latter is rare given the multi-stage verification, but neither direction can be fully ruled out without ground-truth injection experiments. This noise adds another reason the null DiD is inconclusive rather than exonerating.

Difficulty mismatch within matched pairs. Subject-by-level matching does not guarantee equal intrinsic difficulty. In the clearest case (Intermediate Algebra L4), the contaminated problem (`math500_0134`: find $h{+}k{+}a{+}b$ for a hyperbola) is a direct formula application, while the matched clean problem (`math500_0495`: find the domain of $f(x){=}(2{-}x)/\log(2{-}\log(x{-}2))$) requires nested logarithm constraint analysis. The model solves the contaminated version in 995 words and

produces a 2,471-word null on the clean one. Fourteen such mismatched pairs were found across both models, meaning the null rate gap is an upper bound on the true contamination effect, not a pure measure of it.

Evaluation pipeline errors. Despite strong overall judge reliability ($\kappa = 0.83$ against human labels), bidirectional errors remain and compress the measured DiD toward zero. On one side, `math500_0301` (Prealgebra L5) was scored incorrect despite the model correctly computing 5.4, because the ground truth string includes "`\text{cents}`" and the judge penalised the missing unit. On the other side, 25 labels were flipped from incorrect to correct, including at least one erroneous flip (`math500_0190`: model output $\frac{37}{49}\approx 0.755$, true answer $\frac{3}{7}\approx 0.429$, accepted as equivalent). Both error types are most common on Asymptote-drawn chart problems and unit-bearing answers; filtering these problem types would reduce evaluation noise in future runs.

9 Qualitative Demonstrations

We illustrate the contamination-induced brittleness with `math500_0193` (Intermediate Algebra L3). On the original problem the model solves correctly in 1,180 words with no self-correction—a direct Rational Root Theorem application. Swapping the coefficients from $(2, 1)$ to $(3, 2)$ breaks the constant-$= 1$ symmetry that made the original tractable; the model correctly lists eight candidate roots but then spirals for 1,775 words into an unresolvable analysis of which roots are achievable under variable coefficients, never boxing an answer. The matched clean problem (`math500_0114`, partial fractions) produces a 730-word trace that includes two independent verification steps absent from the contaminated trace—consistent with the aggregate -55% self-correction finding. Full traces for all three conditions are in Appendix F.

Contaminated problem (number swap) — *null output*.

> **Problem:** How many different possible rational roots does $3x^4 + a_3x^3 + a_2x^2 + a_1x + 2 = 0$ have, where a_3, a_2, a_1 are integers? **Ground truth: 8.**
>
> **Model (excerpt):** *"... the possible rational roots would be $\pm 1, \pm\frac{1}{3}, \pm 2, \pm\frac{2}{3}$. Wait, that's 8 possible roots... but maybe some could be excluded based on a_3, a_2, a_1? ...for a given r, can we find integers a_3, a_2, a_1 such that substituting $x{=}r$ gives zero? ... Since a_3 must be an integer..."* [*continues 1,500+ more words; no answer produced*]

10 Conclusion

We presented a difference-in-differences framework for causally attributing LLM benchmark score inflation to training data contamination, applied to two open-weight models on MATH-500. Neither model shows a statistically significant DiD effect at the current sample size, and our estimates of benchmark score inflation are small (≤ 0.4 pp) and statistically indistinguishable from zero. However, several secondary signals point toward a contamination-induced behavioral shift in OpenThoughts: a nearly doubled null rate on contaminated number-swap problems (37.5% vs. 20.8%), shorter chain-of-thought traces with markedly fewer self-corrections (-55%) and uncertainty markers (-34%), and earlier answer prediction in the model's forward pass. Together, these suggest that contamination in OpenThoughts manifests not as verbatim memorization but as a subtler shortcut—a bias toward confident, direct solution retrieval that is brittle to superficial numerical changes. The Tulu model shows no detectable signal across any measure, consistent with its lower baseline performance and shorter, less structured reasoning traces. We release our perturbation pipeline, causal evaluation framework, and curated clean baseline to support future contamination audits of open and closed LLMs.

Broader Impact. Benchmark contamination poses a systemic risk to LLM evaluation validity: if models are trained on the very data used to assess them, reported capabilities may overstate true generalization. Our DiD framework provides a principled, model-agnostic method for estimating the causal magnitude of this effect, requiring only black-box inference access and a verified clean comparison set. While our point estimates of benchmark score inflation are small (≤ 0.4 pp), the confidence intervals do not rule out effects as large as ~ 1.8 pp—meaningful on a 500-problem

benchmark. We recommend that future benchmark releases include a clean verification split, and that model evaluation reports publish DiD-adjusted accuracy alongside raw scores as standard practice.

REFERENCES

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. 2020.

Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium*, 2021.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. Investigating data contamination in modern benchmarks for large language models. *arXiv preprint arXiv:2311.09783*, 2024.

Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvarna, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik Sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah Pratt, Vivek Ramanujan, Jon Saad-Falcon, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A. Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alexandros G. Dimakis, and Ludwig Schmidt. OpenThoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.

Zirui He, Haiyan Zhao, Ali Payani, and Mengnan Du. MemLens: Uncovering memorization in LLMs with activation trajectories. *arXiv preprint arXiv:2509.20909*, 2025.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Muhammed Yusuf Kocyigit and Caglar Yildirim. The impact of post-training on data contamination. *arXiv preprint arXiv:2601.06103*, 2026.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengding Hu, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Ximing Lu, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2025.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Emmanuel Candès, Tatsunori Hashimoto, and Percy Liang. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.12599*, 2025.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

Weijia Shi, Anibal Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789*, 2023.

Yifan Sun, Han Wang, Dongbai Li, Gang Wang, and Huan Zhang. The emperor's new clothes in benchmarking? A rigorous examination of mitigation strategies for LLM benchmark data contamination. *arXiv preprint arXiv:2503.16402*, 2025.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddartha Naidu, Chinmay Hegde, Niv Cohen, Nolan Miller, Tom Goldstein, Soheil Feizi, and Abhinav Bhatele. LiveBench: A challenging, contamination-free LLM benchmark. *arXiv preprint arXiv:2406.19314*, 2024.

Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking benchmark and contamination for language models with rephrased samples. *arXiv preprint arXiv:2311.04850*, 2023.

Michał Zawalski, Meriem Boubdir, Klaudia Bałazy, Besmira Nushi, and Pablo Ribalta. Detecting data contamination in LLMs via in-context learning. *arXiv preprint arXiv:2510.27055*, 2025.

A Perturbation Prompts

Number Swap (GPT-4o system prompt).

> You are a math problem editor. Change the numerical values in the problem below to different but similarly-scaled numbers. Rules:
>
> - Do NOT make the problem conceptually harder or easier (same math operations required)
> - Keep the same structure and question type
> - The problem must remain well-defined and have a clean answer
> - Also solve the modified problem and provide the correct answer
> - Rate your confidence in the answer: "high", "medium", or "low"
>
> Return ONLY valid JSON with exactly these keys: `"perturbed_problem"`, `"perturbed_answer"`, `"confidence"`

Surface Noise (GPT-4o system prompt).

> You are a math problem editor. Add surface-level noise to the problem below. Rules:
>
> - Do NOT change the mathematical structure, numerical values, wording, or meaning in any way
> - Add exactly one non-mathematical context sentence (e.g. a setting, a character's motivation, or a real-world scenario) that is irrelevant to solving the problem
> - Do not rephrase, reorder, or alter any existing part of the problem
> - The answer must remain EXACTLY the same as the original
> - Do NOT include the answer, any numerical result, or any hint of the solution in `"perturbed_problem"`
>
> Return ONLY valid JSON with exactly this key: `"perturbed_problem"`

B Full CoT Feature Table — OpenThoughts

C Mechanistic Probing by Subject × Level

Table 9 reports per-bucket attention entropy and logit lens rank for all 18 (Subject, Level) combinations evaluated on OpenThoughts. Each row averages over at most 2 forward passes (contaminated and clean) for that bucket; single-pass variance is high. Entropy_Diff and Rank_Diff are Clean − Contaminated; positive values indicate clean traces show higher entropy / later answer prediction (consistent with more generative reasoning).

Entropy differences are inconsistent in sign across subjects (12 of 18 rows are positive but 6 are negative), and rank differences span from $-37{,}542$ to $+58{,}073$. The average rank difference of

Table 8: CoT features for OpenThoughts across all split × perturbation conditions. TTR = type-token ratio.

Split	Perturbation	N	Words	Uncert.	Hedging	Self-corr.	Math%	TTR
Clean	Number swap	24	1,465	2.29	1.92	0.42	41.8%	0.255
Clean	Original	24	1,251	2.00	1.17	0.38	36.8%	0.263
Clean	Surface noise	24	1,313	1.75	1.88	0.29	39.0%	0.265
Contaminated	Number swap	24	1,439	1.33	1.58	0.38	44.0%	0.263
Contaminated	Original	24	1,160	1.33	0.67	0.17	45.2%	0.284
Contaminated	Surface noise	24	1,277	1.50	1.25	0.17	41.5%	0.277

Table 9: Layer-averaged attention entropy and logit lens rank of the correct answer token by subject × level (OpenThoughts). Positive Entropy_Diff and Rank_Diff indicate clean > contaminated, i.e., more diffuse attention and later answer prediction for clean problems. Results are from single forward passes ($N = 1$) and are highly variable; do not over-interpret individual rows.

Subject	Level	Contam Entropy	Clean Entropy	Entropy Diff	Contam Rank	Clean Rank	Rank Diff
Algebra	1	1.22	1.42	+0.20	9,769	34,352	+24,583
Algebra	3	1.23	1.43	+0.20	21,737	29,011	+7,274
Algebra	4	1.41	1.45	+0.04	35,892	15,024	−20,868
Algebra	5	1.39	1.27	−0.12	1,529	3,608	+2,079
Counting & Prob.	4	1.26	1.42	+0.16	6,365	20,525	+14,160
Counting & Prob.	5	1.38	1.33	−0.04	16,047	58,562	+42,515
Geometry	2	1.36	1.54	+0.18	50,411	16,688	−33,723
Geometry	5	2.30	1.94	−0.36	13	5,158	+5,145
Intermediate Algebra	1	1.26	1.40	+0.14	784	58,857	+58,073
Intermediate Algebra	2	1.47	1.52	+0.05	46,789	9,246	−37,542
Intermediate Algebra	3	1.46	1.55	+0.09	1,425	12,736	+11,312
Intermediate Algebra	4	1.63	1.51	−0.12	6,056	18,497	+12,441
Intermediate Algebra	5	1.59	1.44	−0.15	4,283	12,533	+8,250
Number Theory	4	1.41	1.41	+0.00	17,743	13,270	−4,473
Prealgebra	2	1.34	1.20	−0.15	2,576	39,748	+37,172
Prealgebra	5	1.92	1.71	−0.21	47,042	17,066	−29,976
Precalculus	2	1.40	1.43	+0.02	40,204	32,715	−7,489
Precalculus	3	1.40	1.61	+0.21	6,812	13,190	+6,378
Mean		1.50	1.51	+0.007	18,182	23,477	+5,295

+5,295 masks extreme variability; the result should not be treated as a robust mechanistic signal at $N = 1$.

D SUPPLEMENTARY FIGURES

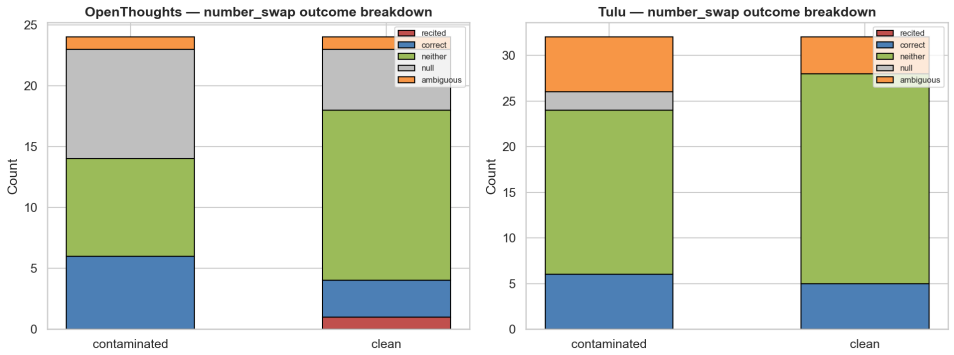

Figure 3: Recitation rate breakdown on number-swap prompts by model and split. Neither contaminated split shows any verbatim recitation of the original answer.

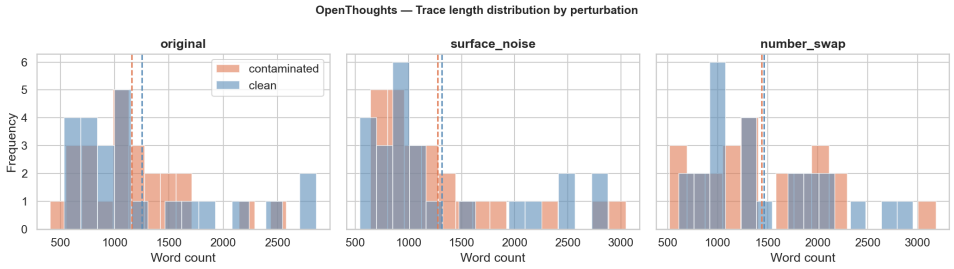

Figure 4: Distribution of CoT trace length (word count) for OpenThoughts by split and perturbation type. Contaminated original traces are slightly shorter; number-swap traces are longer in both splits as the model works harder on the altered problem.

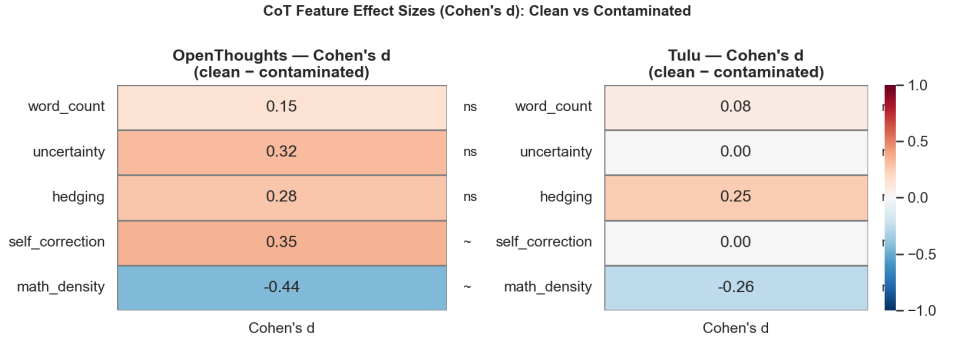

Figure 5: Cohen's d effect sizes for CoT features comparing contaminated vs. clean splits (OpenThoughts). Self-correction and math token density show the largest effects; all remain small-to-medium in magnitude.

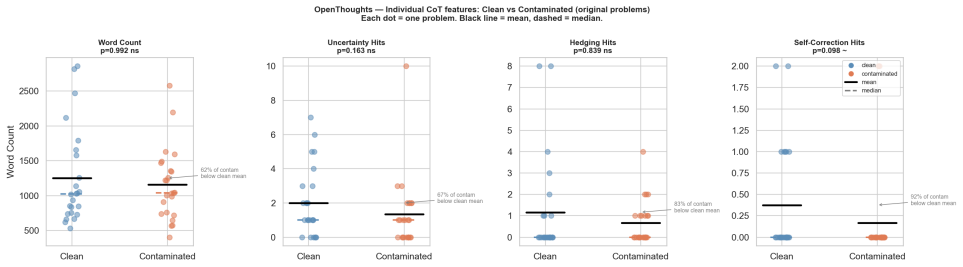

Figure 6: Per-trace CoT feature values for OpenThoughts across all split × perturbation conditions. High within-group variance illustrates why aggregate effects do not survive multiple-comparison correction.

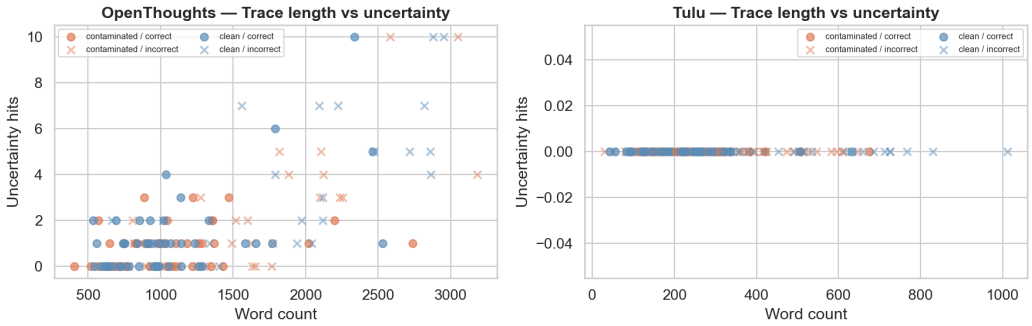

Figure 7: Trace length vs. uncertainty marker count by split and correctness (OpenThoughts). Longer traces correlate with more uncertainty markers; incorrect responses are disproportionately long.

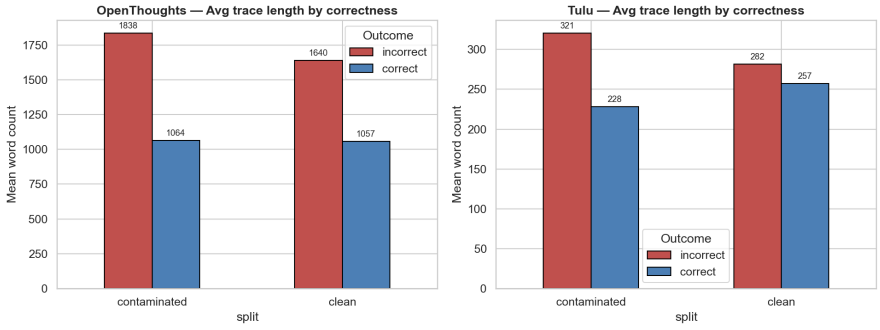

Figure 8: Mean trace length by split and correctness. Incorrect answers produce traces 60–70% longer than correct ones, consistent with the model iterating unsuccessfully rather than finding a direct solution.

Table 10: Estimated benchmark score inflation projected to MATH-500. Inflation $= (N_{\text{contaminated}}/500) \times \text{DiD}$. Assumes the evaluated contaminated problems are representative of the full contaminated fraction.

Model	Perturbation	DiD	Inflation (pp)	95% CI (pp)
OpenThoughts	Number swap	-0.083	-0.40	$[-1.80, +1.00]$
OpenThoughts	Surface noise	$+0.083$	$+0.40$	$[-0.60, +1.60]$
Tulu	Number swap	$+0.031$	$+0.20$	$[-1.40, +1.80]$
Tulu	Surface noise	-0.031	-0.20	$[-1.80, +1.40]$

E Additional Results

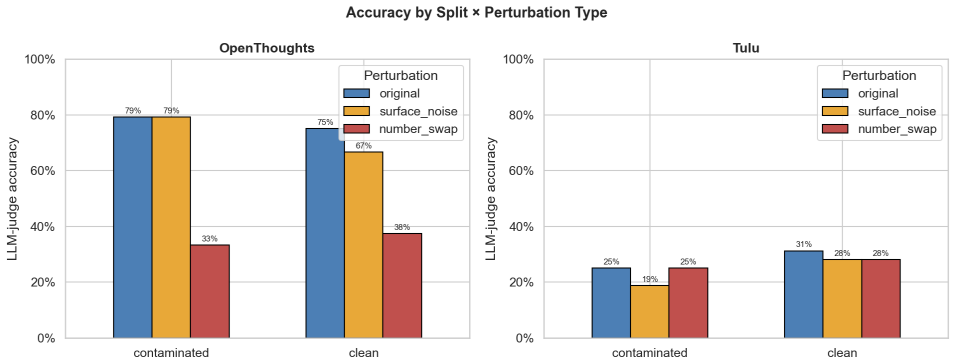

Figure 9: Accuracy by model, split, and perturbation type (LLM-as-judge). Number swap floors both models to ∼30–40% regardless of split; surface noise preserves accuracy for OpenThoughts clean but not contaminated.

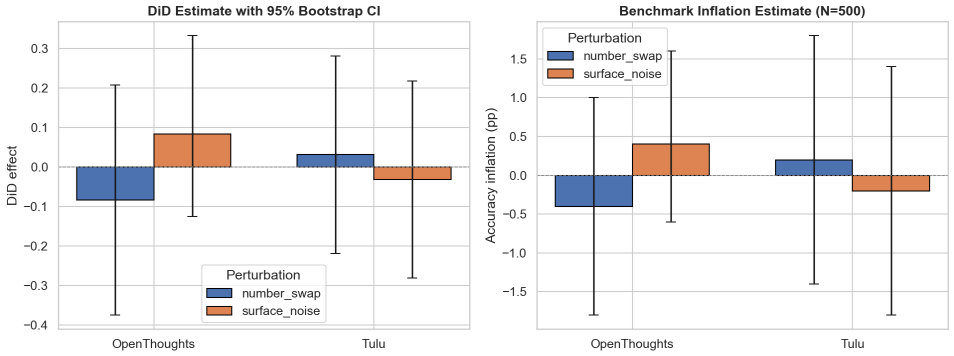

Figure 10: DiD point estimates with 95% bootstrap confidence intervals (left) and projected benchmark score inflation on MATH-500 (right). All intervals span zero; point estimates are $\leq \pm 0.4$ pp.

F Qualitative Demonstrations (Full Traces)

Problem (contaminated, original).

> How many different possible rational roots does $2x^4 + a_3x^3 + a_2x^2 + a_1x + 1 = 0$ have, where a_3, a_2, a_1 are integers? **Ground truth: 4.**

OpenThoughts response (contaminated, original) — 1,180 words, correct.

Table 11: Recitation of original answer on number-swap prompts. "Null" = no \boxed{} extracted; "Ambiguous" = answer extracted but indeterminate.

Model	Split	Total	Null	Ambiguous	Evaluable	Recitations
OpenThoughts	Contaminated	24	9	1	14	0 (0.0%)
OpenThoughts	Clean	24	5	1	18	1 (5.6%)
Tulu	Contaminated	32	2	6	24	0 (0.0%)
Tulu	Clean	32	0	4	28	0 (0.0%)

> *"Okay, so I need to find the number of different possible rational roots . . . From the Rational Root Theorem . . . the possible values of p (the numerator) are the divisors of 1, which are just ± 1. The possible values of q (the denominator) are the divisors of 2, which are $\pm 1, \pm 2$. Therefore, combining these, the possible rational roots should be $\pm 1, \pm \frac{1}{2}$. . . . So that's four distinct possibilities. . . . Since the leading coefficient is fixed at 2 and the constant term is fixed at 1, the possible rational roots can't change based on the other coefficients. Therefore, the number of different possible rational roots is* $\boxed{4}$*."*

The trace is direct: the model applies the theorem, enumerates four candidates, briefly considers whether variable coefficients matter, and resolves the question in under 1,200 words with no self-correction.

Problem (contaminated, number swap).

> How many different possible rational roots does $3x^4 + a_3x^3 + a_2x^2 + a_1x + 2 = 0$ have, where a_3, a_2, a_1 are integers? **Ground truth: 8. Model answer: *null*.**

OpenThoughts response (contaminated, number swap) — 1,775 words, null output.

> *". . . the possible rational roots would be $\pm 1, \pm \frac{1}{3}, \pm 2, \pm \frac{2}{3}$. Wait, that's 8 possible roots. But the question says 'different possible rational roots'. So does each of these fractions count as distinct? . . . However, the problem mentions 'different possible rational roots', but maybe some of these could be excluded based on the coefficients a_3, a_2, a_1? Wait, hold on. The original polynomial has variable coefficients. So the coefficients aren't fixed. Therefore, depending on the values of a_3, a_2, a_1, different roots might be possible or not. . . . In other words, for a given rational number r, can we find integers a_3, a_2, a_1 such that substituting $x = r$ gives zero? Alternatively, rearranging: $a_3 = \frac{-3r^4 - a_2r^2 - a_1r - 2}{r^3}$. Since a_3 must be an integer . . . "*
> [*reasoning continues for 1,500+ more words; no final answer produced*]

The swap changes the leading coefficient from 2 to 3 and constant from 1 to 2. This breaks the constant-$= 1$ symmetry, producing a combinatorially richer candidate set. The model correctly identifies eight candidates but spirals into an unresolvable analysis of achievable roots under variable coefficients.

Problem (clean, original) — matched subject and level.

> $\frac{C}{x-3} + \frac{D}{x+8} = \frac{4x-23}{x^2+5x-24}$ for all $x \neq 3, -8$. Find CD. **Ground truth: -5.**

OpenThoughts response (clean, original) — 730 words, correct.

> *"Okay, so I need to find the product CD . . . Hmm, let's start by understanding what this problem is asking. First, I know that the denominators on both sides must be the same . . . Let me factor the denominator on the right . . . That means the right-hand side is $(4x-23)/[(x-3)(x+8)]$, which matches the denominators on the left side. Great, so that checks out. . . . Therefore, we can set up the following system of equations: (1) $C + D = 4$. (2) $8C - 3D = -23$. . . . So, $C = -1$ and $D = 5$. . . . Wait, let me verify by plugging back into the original equation . . . Compute numerator: $-x - 8 + 5x - 15 = 4x - 23$. Which matches the right side. Perfect! So the solution is correct."*

The clean trace is 450 words shorter but contains two explicit verification steps ("Great, so that checks out" and the plug-back check) absent from the contaminated trace—consistent with the -55% self-correction finding in the aggregate CoT analysis.